



Quick Sort

Dr. Obed Noé Sámano Abonce



Sorting Algorithms

Last time

- Merge sort
 - Split by position (always in the middle)
 - Hard work is done in the merge part.
 - Always $O(n \log n)$

Sorting Algorithms

Today

- Quick sort
 - Divide and conquer again.
 - Split by value!
 - The divide part does the hard work now.
 - There are cases in which it can be as bad as bubble sort.
 - Yet, it is one of the most widely used sorts in practice.

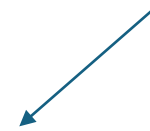
Quick Sort Algorithms

The idea

- **Divide**

- Pick a **pivot**.
- Partition the array:

Pivot will be in its position once the array is sorted



elements < pivot	pivot	elements > pivot
----------------------------	--------------	----------------------------

- **Conquer**

- Recursively quicksort the left and right part.

- **Combine**

- Nothing to do, once both sides are sorted, the whole array is sorted.

Quick Sort Algorithms

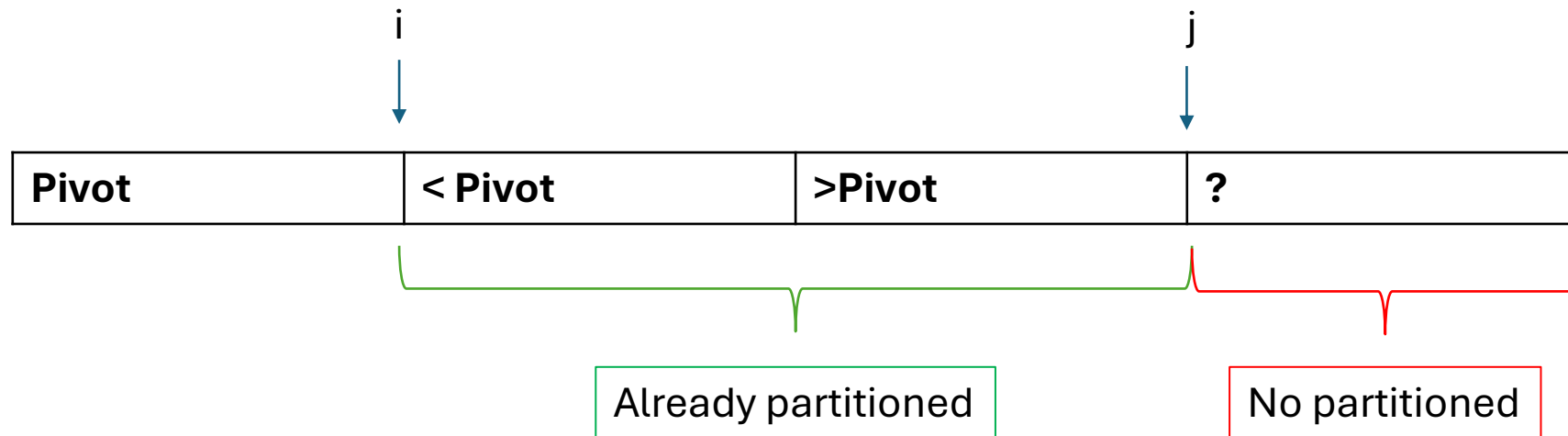
Variations

How to select the pivot and do the partition?

- Hoare classic
 - Pivot $\leftarrow$ first element
 - Two indices converge from opposite ends
- Lomuto classic
 - Pivot $\leftarrow$ last element
 - One index swaps left to right
- Lomuto with pivot at the start
 - Pivot $\leftarrow$ First element
 - One index sweeps left to right

Quick Sort

Partition via Lomuto with the pivot at the start



Quick Sort

Lomuto Partition - Example



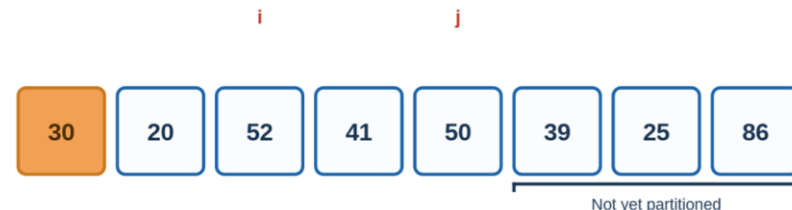
30 will be our pivot. Indices *i* and *j* are both initialized at the starting position + 1.



We advance *j*; 20 joins the partitioned section. Since it is less than the pivot, it is swapped with the element at *i*, and then *i* advances.



We advance *j* and 41 joins the partitioned section.



We advance *j* and 50 joins the partitioned section.



We advance *j* and 39 joins the partitioned section.



We advance *j*; 25 joins the partitioned section. Since it is less than the pivot, it is swapped with the element at *i*, and then *i* advances.



We advance *j* and 86 joins the partitioned section. Since there are no elements left to examine, the pivot is swapped with the element at *i* - 1 to place it in its final position.

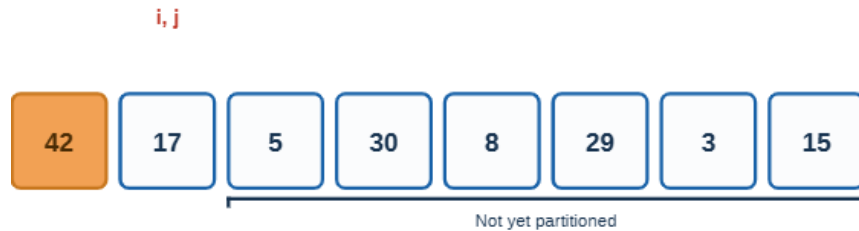


Quick Sort

Lomuto Partition - Example



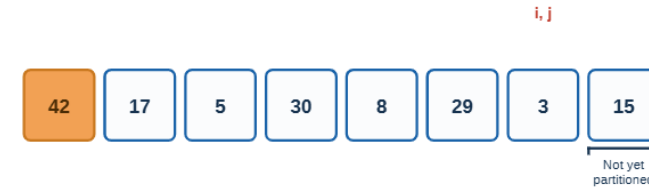
42 will be our pivot. Indices i and j are both initialized at the starting position + 1.



We compare $a[1] = 17$ against the pivot: $17 < 42$, so it joins the partitioned section. Since i and j are already at the same position, swapping has no visible effect — both simply advance to position 2.



We compare $a[2] = 5$ against the pivot: $5 < 42$, so it joins the partitioned section. Since i and j are already at the same position, swapping has no visible effect — both simply advance to position 3.



We compare $a[6] = 3$ against the pivot: $3 < 42$, so it joins the partitioned section. Since i and j are already at the same position, swapping has no visible effect — both simply advance to position 7.

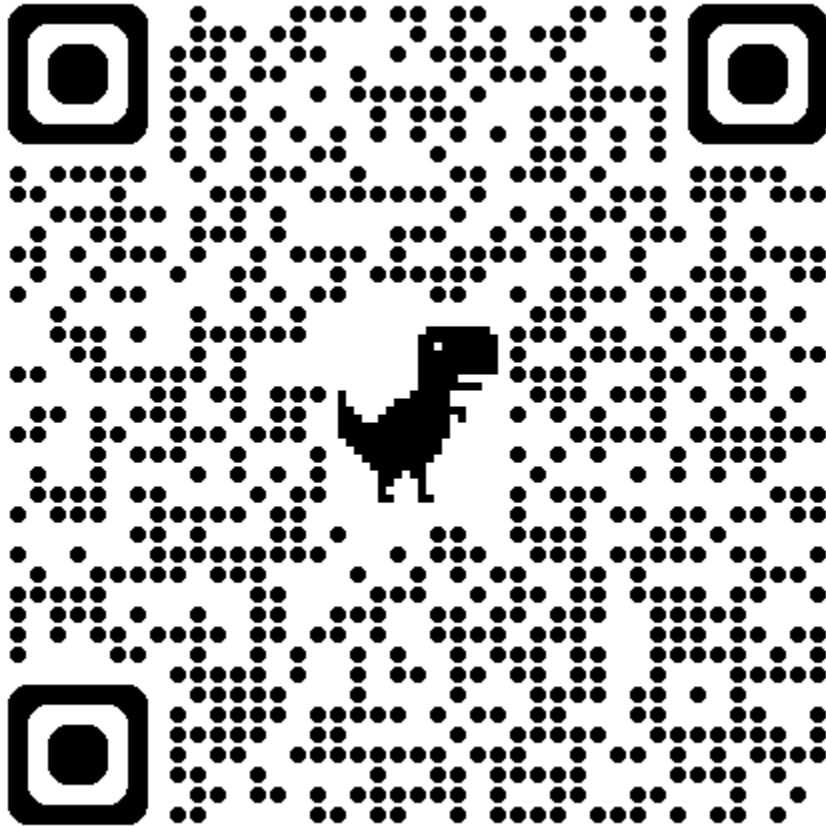


We compare $a[7] = 15$ against the pivot: $15 < 42$, so it joins the partitioned section. Since i and j are already at the same position, swapping has no visible effect — both simply advance to position 8. Since there are no elements left to examine, the pivot is swapped with the element at $i - 1 = 7$ to place it in its final position.



Quick Sort

Visualization



<https://noe-tec.github.io/data-structures/01-fundamentals/pages/quicksort-recursion-tree-lomuto-book.html>

Quick Sort

Comparisons and swaps

42	17	5	30	8	29	3	15
----	----	---	----	---	----	---	----

The full run

Call	Range · pivot	comparisons	swaps	pivot lands at
1	a[0..7] · 42	7	1	7
2	a[0..6] · 15	6	4	3
3	a[0..2] · 3	2	0	0
4	a[1..2] · 5	1	0	1
5	a[4..6] · 17	2	0	4
6	a[5..6] · 29	1	0	5
Total		19	5	

Sorted. [3, 5, 8, 15, 17, 29, 30, 42]

Quick Sort

The code

```
int partition(int a[], int lo, int hi) {  
    int pivot = a[lo];           // first element of the range  
    int i = lo + 1;              // boundary of the < pivot zone  
    for (int j = lo + 1; j <= hi; j++) {  
        if (a[j] < pivot) {  
            int temp = a[i]; a[i] = a[j]; a[j] = temp;  
            i++;  
        }  
    }  
    int temp = a[lo]; a[lo] = a[i - 1]; a[i - 1] = temp;  
}
```

quickSort.cpp

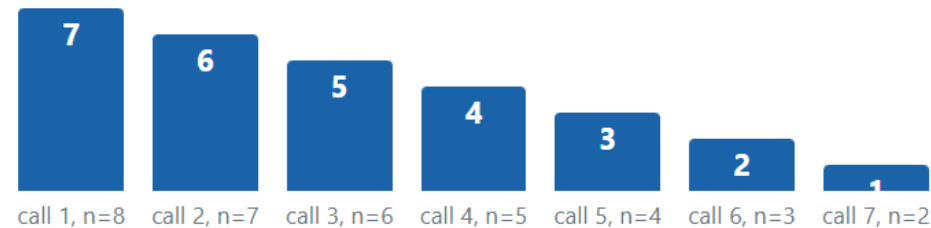
```
void quickSort(int a[], int lo, int hi) {  
    if (lo >= hi) return;        // 0 or 1 element: already sorted  
    int p = partition(a, lo, hi);  
    quickSort(a, lo, p - 1);  
    quickSort(a, p + 1, hi);  
}
```

Quick Sort

Complexity – Worst case

Worst Case → The partition is unbalanced

- Imagine we have the bad luck of always picking the minor element as the pivot. When may this happen?
- Every partition splits off just one element (itself) and keeps everything else on one side.



$n-1$ recursive calls deep, instead of $\log_2 n$ — one side is empty every single time

$$(n-1) + (n-2) + \dots + 1 = n(n-1)/2 \rightarrow \Theta(n^2)$$

Quick Sort

Complexity – Best case

Best case → The partition is balanced

Assuming that we always pick the middle element

Partitioning level	Partitions	Size each	Comparisons this level
Level 0 · whole array	1	n	n
Level 1	2	n/2	$2 \cdot (n/2) = n$
Level 2	4	n/4	$4 \cdot (n/4) = n$
...	...	...	...

} $\log_2 n$

$$\underbrace{n + n + \dots + n}_{\log_2 n} = n \log_2 n$$

Quick Sort

Complexity – Best case

Best case → The partition is balanced

- If the pivot happens to land near the middle each time, quicksort makes **2** calls, each on roughly $n/2$ elements, plus **$O(n)$** of partitioning work:

$$T(n) = 2 \cdot T(n/2) + O(n)$$

The same Master Theorem box from merge sort: $T(n) = a \cdot T(n/b) + O(n^d)$ — compare a to b^d		
Compare	Result	
$a > b^d$	$O(n^{\log_b a})$	
$a = b^d$	$O(n^d \cdot \log n)$	← balanced quicksort
$a < b^d$	$O(n^d)$	

Same $a = 2$, $b = 2$, $d = 1$ as merge sort → **$O(n \log n)$**

Quick Sort

Complexity – Best case

	Split	Order of growth
Worst case	always 1 and $n-1$	$\Theta(n^2)$
Average case	roughly balanced, on random input	$O(n \log n)$
Best case	always in half	$\Theta(n \log n)$

Quick Sort

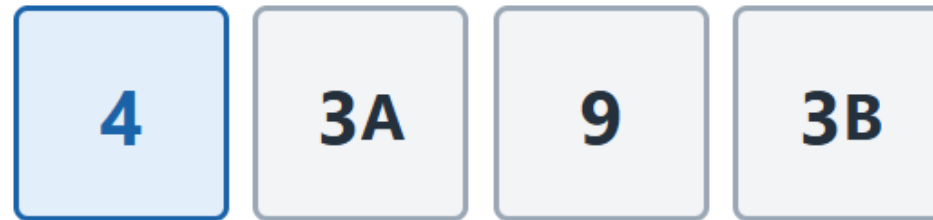
In place, unlike merge sort

- Merge sort's merge step needed a **temporary array** to build its output — extra space proportional to n .
- Quicksort's partition only swaps elements inside the array itself. **No temporary array**, no extra space beyond a few local variables.

Quick Sort

Stability

BEFORE · PIVOT = 4 (FIRST ELEMENT)



AFTER · 3 COMPARISONS, 2 SWAPS



No stable

Quick Sort

Comparison

	Worst case	Best case	Stable?	In place?
Bubble sort	$\Theta(n^2)$	$O(n)$	yes	yes
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	no	yes
Insertion sort	$\Theta(n^2)$	$O(n)$	yes	yes
Merge sort	$\Theta(n \log n)$	$\Theta(n \log n)$	yes	no
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	no	yes

No single algorithm wins on every column — which is exactly why real libraries do not pick just one.

Quick Sort

Take aways

- Quicksort divides by value not by position.
- Best case $O(n \log n)$, worst case $O(n^2)$
- It sorts in place, with no extra array (with the trade-off for not guaranteeing the good case).
- It is not stable: a swap can reorder two equal elements that were never directly compared.

Quick Sort

Selecting the pivot

- Quicksort's real-world efficiency comes down to pivot quality.
- Common ways to pick it: the first element, the last element, a random element, or the median of three (first, middle, and last).
- Experiments show Quicksort runs close to $O(n \log_2 n)$ whenever each split falls anywhere between 25% / 75%.
- Picking at random usually works well in practice.
- A linear time choosing strategy does not make sense.

References

M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed.
Boston, MA, USA: Pearson, 2014. Chapter 2.

V. M. de la Cueva Hernández, L. H. González Guerra, y E. G. Salinas
Gurrión, *Estructuras de datos y algoritmos fundamentales*.
Monterrey, México: Editorial Digital del Tecnológico de Monterrey,
2020.